

Tiered Customer Matching

Voice Agent → ServiceTrade Webhook | Question flow, tier rules & confidence tuning

The core idea. When a call comes in, the system identifies the customer/location by comparing what the caller said (phone, address, business/location name, company) against ServiceTrade records. Because speech-to-text is imperfect, every comparison is **fuzzy**. Each candidate location is assigned a **confidence tier (1, 2, or 3)** based on how many independent signals agree, and the tier decides whether a job is auto-created, GPT-disambiguated, or sent to a human for review.

A. The Questions — in order to ask

Ask the **fewest** questions needed and **stop the moment a Tier 1 match is reachable**. Fields are ordered by *closing power* — how reliably a match on that field alone (or with what's already collected) lands a confident match.

#	Ask the caller...	Why it closes the match	Tier 1 rule it triggers
0	(nothing — phone from caller ID)	Phone is captured automatically. If it maps to exactly one location, the match is done with zero questions.	phone_match_single_location
1	Service address	Phone + address is the strongest pairing; addresses are highly unique.	phone_and_address_match
2	Business / location name	Pairs with the address just given. Rescues callers whose phone isn't on file.	location_name_and_address_match
3	Best callback / direct number	Used as the <i>fallback phone</i> — triggers a second matching pass. Can solo-close if single-location.	phone_match_single_location
4	Company name	Tiebreaker only. By design it can never reach Tier 1 alone (STT confusion, e.g. "Uptown" vs "Intown").	Feeds GPT disambiguation

Always collect the caller's name + a callback number for the job record — even though the name carries *no* tier weight (it's not used in scoring, only in the job description and the "is this call actionable?" gate).

Early-exit logic for the conversation flow

```
phone (free) → single location? → STOP · Tier 1
  else ask ADDRESS → matches with phone? → STOP · Tier 1
    else ask BUSINESS/LOCATION ▶ matches with address? → STOP · Tier 1
      else ask CALLBACK NUMBER ▶ single location? → STOP · Tier 1
        else ask COMPANY NAME ▶ hand to GPT / human review
```

Important: the webhook scores *whatever the call collected* — it has no notion of "stop asking." The early-exit must live in the **Retell conversation flow** (a mid-call function that re-checks the match after each answer). Without that, all questions are asked regardless, and the ordering only decides which the caller hears first.

B. The Tiered Matching

Every candidate location gets the **first rule that matches**, evaluated strongest-first. A tier reflects how many independent signals corroborate the match.

TIER 1 High confidence → auto-create job

Two strong independent identifiers agree, or one provably-unique identifier.

Rule	What agrees
phone_and_address_match	phone + address
location_name_and_address_match	location name + address
phone_match_single_location	phone, and it maps to only 1 location
company_location_and_address_exact	company + location + address all exact
location_and_company_exact	location + company exact (no address needed)
phone_match_with_address_similarity	phone + address similarity > 0.5

Action: take the best-ranked Tier 1 candidate → cross-validate → create job. No human review.

TIER 2 Medium confidence → create if unambiguous, else review

One solid signal, or fuzzy-but-plausible combinations. Company+address (no location) lives here on purpose.

Rule	What agrees
phone_match_multiple_locations	phone, but maps to 2+ locations
company_and_address_exact_no_location	company + address, no location confirm
location_name_exact	location name alone
company_fuzzy_and_address	company fuzzy > 0.6 + address
location_fuzzy_and_address	location fuzzy > 0.8 + address
company_and_location_fuzzy	both fuzzy > 0.8

Action: if all Tier 2 candidates point to the **same** location → create job + review note. If they differ → **GPT disambiguation**; if unresolved → manual review (`multiple_medium_confidence_matches`).

TIER 3 Low confidence → never auto-create, always review

A single weak signal with nothing corroborating: company exact alone, location fuzzy > 0.7 alone, address alone. No signals at all → **no_strong_match**.

Action: no job. Manual review (**low_confidence_matches**) with the top 3 candidates attached to the notification email.

How the tiers gate the whole flow

```
findCustomerWithConfidence() → ranked candidates
|
|─ 0 candidates → REVIEW (no_matches)
|
|─ any Tier 1? – yes → tier1[0] → cross-validate
|                       |─ pass → CREATE JOB
|                       |─ fail → REVIEW (retell_data_mismatch)
|
|─ any Tier 2? – yes → same location?
|                       |─ yes → CREATE JOB + note
|                       |─ no  → GPT disambiguation
|                               |─ resolved → CREATE JOB
|                               |─ no      → REVIEW (multiple_medium_confidence_matches)
|
|─ else (Tier 3) → REVIEW (low_confidence_matches)
```

C. Setting up the confidence scores

Step 1 — Normalize before comparing

All comparisons run on normalized strings, so casing/punctuation never cause a miss.

- `normalizePhone()` — strip everything but digits. "(214) 555-1234" → "2145551234"
- `normalizeText()` — lowercase, drop punctuation, collapse whitespace. "Dallas Fire & Alarm, LLC" → "dallas fire alarm llc"

Step 2 — Compute a 0–1 similarity per field

Two algorithms run; `fuzzySimilarity()` takes the higher of the two (`Math.max`):

Algorithm	How it scores	Best at catching
Token (Jaccard)	shared words ÷ total unique words	word-order / extra-word differences
Bigram (Dice)	shared 2-char chunks, doubled ÷ total chunks	character-level typos ("Diversetec" vs "DIVERSATEK")

Address gets special handling: exact → 1.0; one contains the other → 0.9; else fuzzy. A stricter *address query match* also tests search variants (with/without "St"/"Street", "N"/"North", unit numbers).

Step 3 — The thresholds you tune

These constants are the dials. Raise them to reduce false matches (more reviews); lower them to auto-create more aggressively (more risk).

Threshold	Used for	Effect of raising it
> 0.95	Company fuzzy → Tier 1 (with location + address)	Fewer fuzzy company auto-creates
> 0.9	Company fuzzy → Tier 2 (with address)	Stricter medium-confidence bar
> 0.8	Location fuzzy + address; company&location both fuzzy	Fewer borderline Tier 2
> 0.75	Address-only strict threshold; cross-validation	Address-only calls harder to match
> 0.7	Min fuzzy to stay Tier 3 (not discarded)	More candidates dropped entirely
> 0.6	Company fuzzy + address; lenient address; cross-val company	Fewer lenient matches
> 0.5	Address similarity + phone → Tier 1	Phone+address pairing stricter
first 5 chars	Company prefix match (STT drift)	n/a — change the prefix length

Address match is adaptive: when phone or location is *also* present, address similarity only needs > 0.6 ; when address is the *only* non-phone identifier it must clear > 0.75 . More corroboration → looser per-field bar.

Step 4 — Two safety gates after scoring

- **Cross-validation:** even a Tier 1 winner must agree with at least one caller-provided field, or it's rejected as `retell_data_mismatch`.
- **Address direct-hit narrowing:** if an address search returns 2+ hits, keep only the one that also matches a name — stops a strong phone match at the wrong address from winning.

Step 5 — Tune from the logs

Every match emits a `matching_summary` log with `searchData`, per-candidate signals, tier counts and the chosen `tierReason`. Use it to:

- Spot fields that were *asked but never scored* (collection gap).
- Find `tierReason` values that auto-create too eagerly (lower confidence → raise that threshold).
- Find frequent `low_confidence_matches` / `no_matches` (callers aren't giving enough → adjust the question flow in Section A).

Recommended starting policy. Keep Tier 1 strict (needs 2 signals or a unique phone). Let Tier 2 lean on GPT disambiguation rather than auto-creating across multiple locations. Treat every threshold change as a trade between *fewer human reviews* and *more wrong-location jobs* — move one dial at a time and watch the `matching_summary` logs for a week before the next change.